



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN

Sistemas Distribuids

Proyecto 3. Servicio de operaciones sobre archivos remotos mediante gRPC

Nombre de los INTEGRANTES

- Ayala Hernández María Fernanda
- Saldaña Navarrete Andrea
- Ugartechea González Luis Antonio
- Tepal Briseño Hansel Yael

NOMBRE DEL PROFESOR: ELBA KAREN SÁENZ GARCÍA
GRUPO: 03
SEMESTRE: 2027-1

FECHA DE ENTREGA: 8 de noviembre del 2026

Objetivo

Familiarizar al alumno en el conocimiento de construcción de máquinas de estados usando direccionamiento de memorias con el método de direccionamiento implícito.

Introducción

En este proyecto se aborda el diseño e implementación de un sistema distribuido fundamentado en llamadas a procedimientos remotos (RPC) mediante la tecnología gRPC. El objetivo principal es establecer un mecanismo de difusión donde un proceso central (Coordinador) envía un mensaje simultáneo a múltiples nodos (Receptores).

A diferencia de un esquema de red tradicional, gRPC abstrae la complejidad de los *sockets*, permitiendo que el coordinador invoque métodos en los receptores remotos como si fueran funciones locales. El proyecto demuestra conceptos fundamentales de sistemas distribuidos, incluyendo la definición de interfaces mediante Protocol Buffers, la tolerancia a fallos parciales (nodos inalcanzables) y el impacto de la concurrencia en los tiempos de respuesta.

Desarrollo e Implementación

1. Preparación del Entorno

Para garantizar un entorno de ejecución aislado y evitar conflictos con otras bibliotecas del sistema operativo, el desarrollo se llevó a cabo dentro de un entorno virtual de Python. Se creó y activó el entorno virtual ejecutando los siguientes comandos en la terminal:

```
python3 -m venv .venv
source .venv/bin/activate
```

Una vez activo el entorno, se procedió a instalar las dependencias oficiales necesarias para habilitar gRPC en el proyecto:

```
python3 -m pip install --upgrade pip
python3 -m pip install grpcio grpcio-tools
```

La biblioteca `grpcio` proporciona el núcleo para establecer la comunicación cliente-servidor en red, mientras que `grpcio-tools` incluye el compilador `protoc`, el cual es indispensable para traducir nuestro archivo de definición de servicios a código Python funcional.

Para verificar que la instalación fue exitosa, se ejecutó el siguiente comando:

```
python3 -c "import grpc; print(f'gRPC versión: {grpc.__version__}')"
```

Si este regresa un número de versión sin errores, significa que gRPC está correctamente instalado y listo para ser utilizado en el proyecto.

2. Definición del Protocolo y Generación de Archivos

El pilar de la arquitectura es el archivo `difusion.proto`, el cual funciona como el contrato estricto entre el cliente (coordinador) y los servidores (receptores). En él se definieron:

- **Mensaje:** Estructura de datos enviada por el coordinador, conteniendo su `emisor_id` y el `contenido` en texto.
- **Confirmacion:** Estructura devuelta por cada receptor, indicando su `receptor_id`, un booleano `recibido`, y un `detalle` textual.
- **Servicio:** El servicio RPC `ServicioDifusion` con el método `RecibirMensaje`.

A partir de este archivo, se utilizó el compilador de Protocol Buffers para generar el código base en Python mediante el siguiente comando:

```
python3 -m grpc_tools.protoc -I. --python_out=. \
        --grpc_python_out=. difusion.proto
```

Esto generó automáticamente los archivos `difusion_pb2.py` (que contiene las clases de los mensajes) y `difusion_pb2_grpc.py` (que contiene el *Stub* para el cliente y el *Servicer* para el servidor).

3. Diagrama de Arquitectura y Tolerancia a Fallos

El sistema utiliza un modelo de cliente-servidor invertido respecto a la web tradicional: aquí los receptores actúan como servidores gRPC (a la escucha continua de mensajes) y el coordinador actúa como el cliente que inicia la conexión.

1. **Receptores en espera:** Los procesos receptores se inician primero y actúan como servidores gRPC estáticos. Se quedan en *standby* escuchando indefinidamente en sus puertos asignados.
2. **Envío de mensajes:** Al iniciar el coordinador (cliente), este ya cuenta con la lista de direcciones. Dispara directamente la llamada remota `RecibirMensaje` con el payload hacia todos los receptores.
3. **Verificación de estado:** El coordinador descubre qué nodos están vivos en el mismo momento de intentar la entrega. Si un receptor responde, se procesa su confirmación. Si está inalcanzable, la conexión expira tras un *timeout* de 2 segundos (lanzando `grpc.RpcError`), marcándolo como caído sin bloquear al resto.

4. **Desconexión:** Una vez recolectadas todas las respuestas, el coordinador presenta el resumen general y finaliza su ejecución.

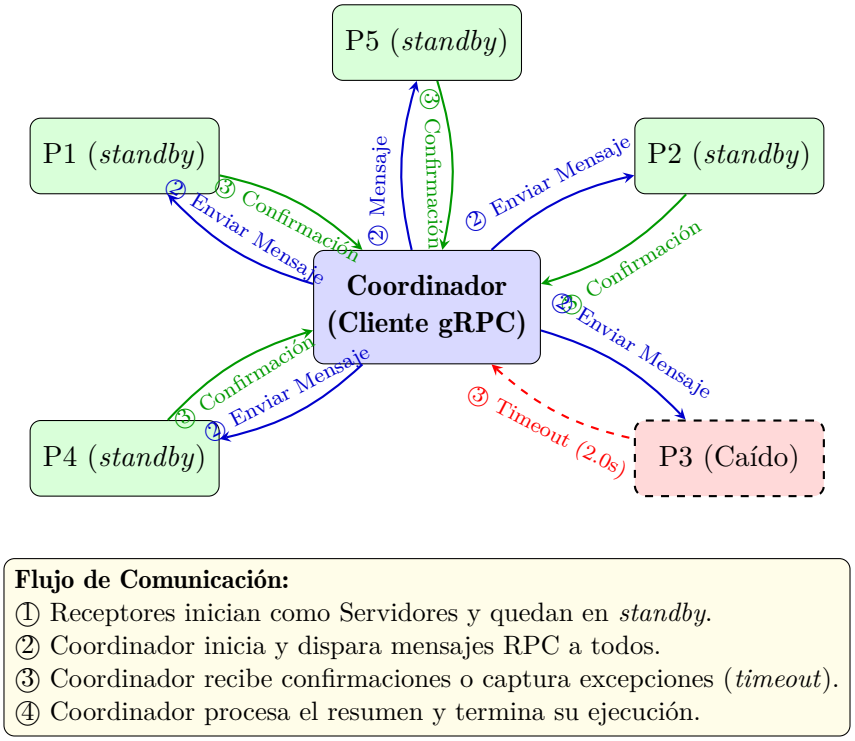


Figure 1: Topología radial del sistema. El coordinador interactúa directamente con los procesos en espera sin fase previa de descubrimiento. La flecha punteada en P3 ilustra la respuesta de *timeout* que marca la tolerancia a fallos.

4. Implementación de los Nodos (Receptores y Coordinador)

El sistema se divide en dos componentes principales, cuyo código en Python responde directamente a los requerimientos funcionales solicitados:

El Receptor (`receptor.py`)

Actúa como un servidor gRPC instanciado mediante la clase `ServicioDifusion`.

Identificación y configuración: A través de los argumentos de la línea de comandos (`sys.argv`), cada receptor recibe dinámicamente su ID único y su puerto de escucha al inicializarse. Esto cumple con el requerimiento de configurar la identificación y dirección de cada receptor sin codificarlos rígidamente (hardcodear) en el código del servidor.

```
if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Uso: python receptor.py <ID_NODO> <PUERTO>")
```

```
        sys.exit(1)
    id_nodo = sys.argv[1]
    puerto_nodo = sys.argv[2]
    iniciar_receptor(id_nodo, puerto_nodo)
```

Inicialización del servidor: La función `iniciar_receptor` toma estos parámetros dinámicos, crea un pool de hilos para poder manejar múltiples peticiones entrantes si fuera necesario, y deja al servicio gRPC en estado de escucha activa.

```
def iniciar_receptor(receptor_id, puerto):
    servidor = grpc.server(futures.ThreadPoolExecutor(max_workers=5))
    servicio = ServicioDifusion(receptor_id)
    difusion_pb2_grpc.add_ServicioDifusionServicer_to_server(servicio, servidor)
    servidor.add_insecure_port(f"[::]:{puerto}")
    servidor.start()
    print(f"Receptor {receptor_id} activo y escuchando en el puerto {puerto}...")
    servidor.wait_for_termination()
```

Recepción y respuesta: La función `RecibirMensaje` extrae el campo `emisor_id` y `contenido` del mensaje entrante para mostrarlo en pantalla de forma local. Inmediatamente después, construye y retorna un objeto `Confirmacion` indicando su estado de éxito (`recibido=True`), cumpliendo así con la devolución de la confirmación solicitada al coordinador.

```
def RecibirMensaje(self, request, context):
    print(f"\n[{self.receptor_id}] Mensaje entrante de {request.emisor_id}:")
    print(f" -> Contenido: {request.contenido}")
    # Devuelve la confirmación requerida
    return difusion_pb2.Confirmacion(
        receptor_id=self.receptor_id,
        recibido=True,
        detalle=f"El nodo {self.receptor_id} procesó el mensaje exitosamente."
    )
```

El Coordinador (`coordinador.py`)

Opera como el cliente central del sistema, encargado de orquestar la difusión masiva.

Configuración dinámica y mapeo: Hace uso de la función `sys.argv` para mapear una cantidad arbitraria de receptores en caso de que se configuren las IP de distintas máquinas a través de la terminal. Si no se ingresan parámetros externos, el coordinador ofrece un modo interactivo para configurar receptores locales en la misma máquina, generando puertos secuenciales automáticamente.

```
direcciones_ingresadas = sys.argv[1:]

if len(direcciones_ingresadas) > 0:
    # Si el usuario proporcionó IPs al ejecutar, las asignamos dinámicamente
    for i, direccion in enumerate(direcciones_ingresadas, start=1):
        RECEPTORES.append({"id": f"P{i}", "direccion": direccion})
else:
    # Fallback local interactivo: Permite configurar 3, 5 o N receptores
    print(" Aviso: No se detectaron IPs externas desde la terminal.")
    try:
        num_receptores = int(input("¿Cuántos receptores locales deseas iniciar (ej. 3 o 5)?: "))
        if num_receptores <= 0:
            raise ValueError
    except ValueError:
        print("Valor inválido. Se configurarán 5 receptores por defecto.")
        num_receptores = 5

    for i in range(1, num_receptores + 1):
        puerto = 50050 + i # Genera puertos secuenciales: 50051, 50052...
        RECEPTORES.append({"id": f"P{i}", "direccion": f"localhost:{puerto}"})
```

Tolerancia a fallos y manejo de errores: La función `contactar_receptor` envuelve la llamada RPC en un bloque `try-except` y añade el parámetro estricto `timeout=2.0`. Si un receptor está inalcanzable, la excepción de red `grpc.RpcError` es capturada de forma segura. En lugar de detener el programa completo, se registra el fallo devolviendo un diccionario con la bandera `"exito": False`, permitiendo presentar un resumen final coherente.

```
def contactar_receptor(receptor, mensaje):
    """Intenta enviar el mensaje a un receptor manejando errores de conexión."""
    try:
        with grpc.insecure_channel(receptor["direccion"]) as canal:
            cliente = difusion_pb2_grpc.ServicioDifusionStub(canal)
            # El timeout evita que el programa se detenga si un nodo cae
            respuesta = cliente.RecibirMensaje(mensaje, timeout=2.0)
            return {"id": receptor["id"], "exito": True, "datos": respuesta}
    except grpc.RpcError as e:
        return {"id": receptor["id"], "exito": False, "error": e.
```

```
            code().name}
```

Impresión de resumen final: Tras completar todas las llamadas RPC, la función `imprimir_resumen` recorre los resultados y presenta un informe detallado de cada receptor, indicando si la difusión fue exitosa o si se produjo un fallo de comunicación.

```python
def imprimir_resumen(resultados, tiempo_ejecucion, tipo_difusion):
    """Muestra el resumen de respuestas e identifica a los procesos caídos."""
    exitosos = [r for r in resultados if r["exito"]]
    fallidos = [r for r in resultados if not r["exito"]]

    print(f"\n--- Resumen de Difusión {tipo_difusion} ---")
    print(f"Tiempo total: {tiempo_ejecucion:.4f} segundos")
    print(f"Confirmaciones: {len(exitosos)} de {len(RECEPTORES)}")

    for exito in exitosos:
        print(f" [OK] {exito['id']}: {exito['datos'].detalle}")

    if fallidos:
        print("\nProcesos que NO respondieron:")
        for fallo in fallidos:
            print(f" [FALLO] {fallo['id']}: Error de comunicación ({fallo['error']})")
    print("-" * 40)
```

Difusión Concurrente vs Secuencial: Para optimizar los cuellos de botella generados por la red, la función `ejecutar_difusion_concurrente` utiliza la librería `concurrent.futures`. Esto permite abrir múltiples conexiones y ejecutar las llamadas RPC de manera simultánea en lugar de iterar nodo por nodo.

```python
def ejecutar_difusion_secuencial(emisor, texto):
    print(f"\nIniciando difusión SECUENCIAL del mensaje: '{texto}'")
    mensaje = difusion_pb2.Mensaje(emisor_id=emisor, contenido=texto)
    resultados = []

    inicio = time.time()
    for receptor in RECEPTORES:
        resultados.append(contactar_receptor(receptor, mensaje))
    fin = time.time()

    imprimir_resumen(resultados, fin - inicio, "SECUENCIAL")

def ejecutar_difusion_concurrente(emisor, texto):
    print(f"\nIniciando difusión CONCURRENTE del mensaje: '{texto}'")
```

```
    mensaje = difusion_pb2.Mensaje(emisor_id=emisor, contenido=texto)
    resultados = []

    inicio = time.time()
    # Usamos ThreadPoolExecutor para lanzar múltiples llamadas RPC en paralelo
    with futures.ThreadPoolExecutor(max_workers=len(RECEPTORES)) as executor:
        tickets = [executor.submit(contactar_receptor, rec, mensaje) for rec in RECEPTORES]
        for ticket in futures.as_completed(tickets):
            resultados.append(ticket.result())
    fin = time.time()

    # Ordenar resultados por ID para que la salida sea legible
    resultados.sort(key=lambda x: x["id"])
    imprimir_resumen(resultados, fin - inicio, "CONCURRENTE")
```

Interacción de usuario: Todo el ciclo de vida del coordinador se encuentra dentro de un ciclo infinito `while True` apoyado por sentencias `input()`. Esto permite mantener el cliente vivo, enviar tantos mensajes distintos como se desee en una misma sesión (validando la actualización en los receptores) y alternar entre los modos secuencial y concurrente a placer.

```
if __name__ == "__main__":
    while True:
        tipo_difusion = input("Elección (Secuencial[0], Concurrente[1]): ")
        if tipo_difusion == "salir":
            break
        elif tipo_difusion == "0":
            texto = input("\nEscribe el mensaje (Secuencial): ")
            ejecutar_difusion_secuencial("Coordinador", texto)
        elif tipo_difusion == "1":
            texto = input("\nEscribe el mensaje (Concurrente): ")
            ejecutar_difusion_concurrente("Coordinador", texto)
```

Manual de Ejecución

Para probar la solución, es imperativo que los procesos receptores (servidores) inicien su ejecución antes que el coordinador (cliente). Todos los comandos asumen que el entorno virtual con las dependencias (`grpcio`) se encuentra activo.

Ejecución Local (Misma Máquina)

Abra 6 terminales en la raíz del proyecto.

1. En las primeras 5 terminales, levante los nodos especificando su ID y puerto:
   - `python3 receptor.py P1 50051`
   - `python3 receptor.py P2 50052`
   - `...  (hasta P5 50055)`
2. En la sexta terminal, inicie el coordinador sin argumentos (usará `localhost`):
   - `python3 coordinador.py`

Ejecución Distribuida (Múltiples Máquinas en Red LAN)

1. Averigüe la IP local de cada máquina receptora (ej. `ip a` en Linux).
2. Ejecute `receptor.py` en cada máquina indicando un puerto disponible.
3. En la computadora del coordinador, ejecute el programa pasando como argumentos las direcciones IP y puertos separados por espacio:
   - `python3 coordinador.py 192.168.1.100:50051 192.168.1.101:50052 ...`

Resultados Obtenidos y Pruebas

1. Difusión exitosa a tres receptores activos

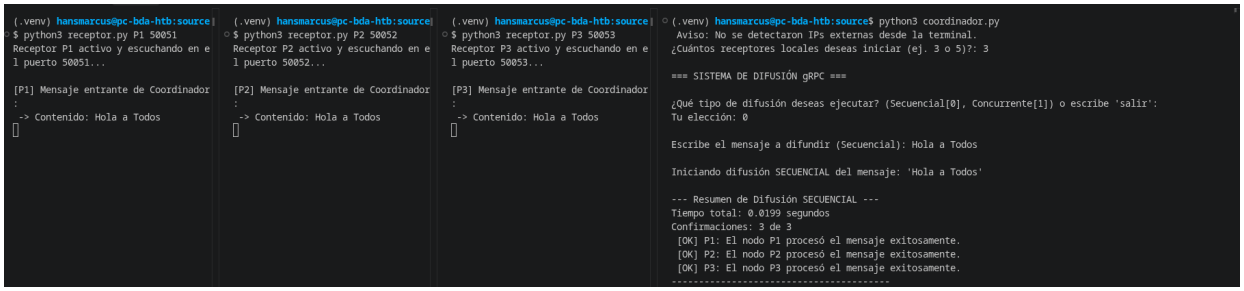


Figure 2: Ejecución de difusión exitosa a tres receptores activos.

Se observa la confirmación de recepción en cada nodo y el resumen final en el coordinador. Esta prueba es ejecutada en modo secuencial, donde cada llamada RPC se realiza una tras otra, resultando en un tiempo de latencia acumulado mayor.

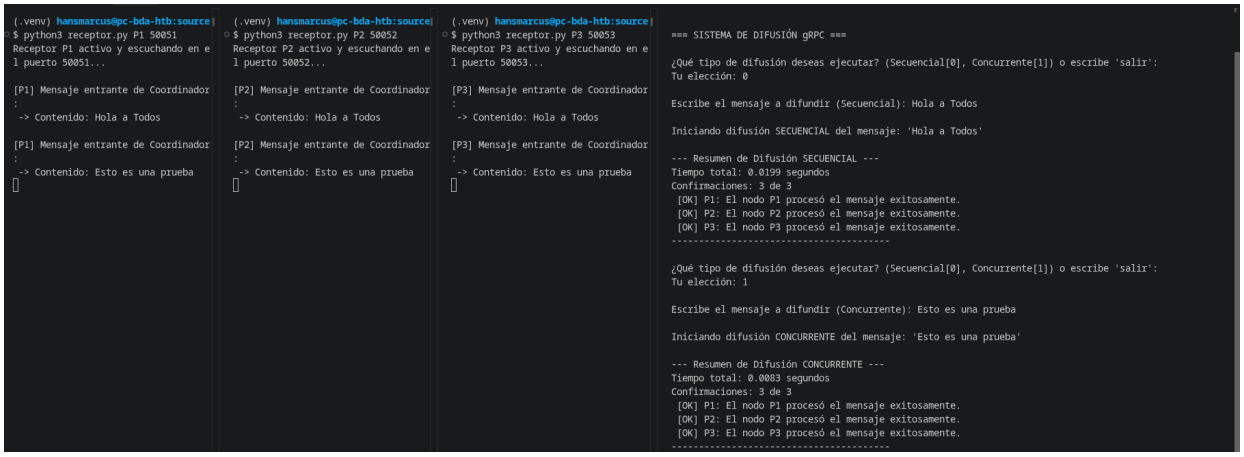


Figure 3: Misma prueba pero en modo concurrente.

Se aprecia la reducción significativa del tiempo de latencia al enviar los RPC en paralelo. Ya que todos los receptores están activos, el coordinador recibe confirmaciones casi simultáneamente, demostrando la eficiencia del enfoque concurrente.

2. Ejecución con un receptor detenido o inaccesible.

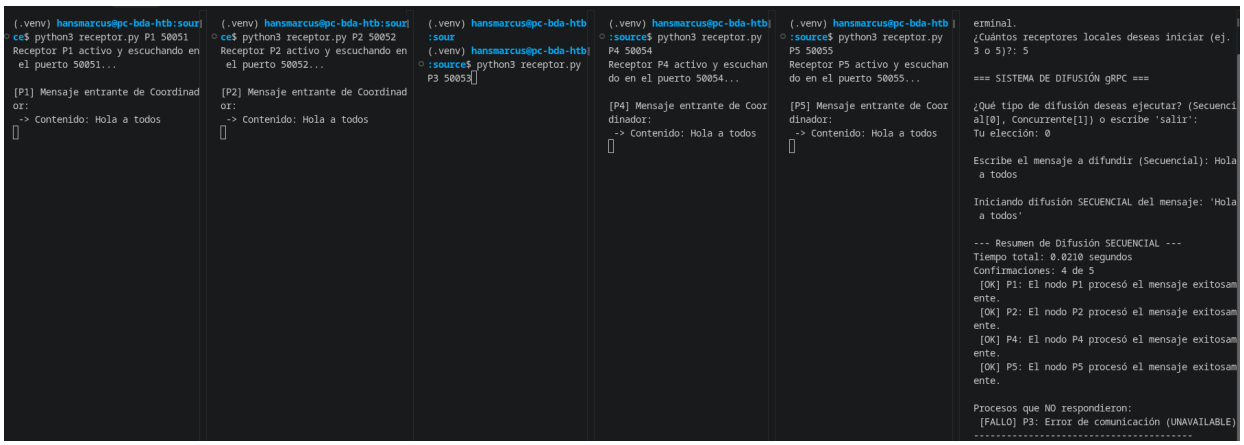


Figure 4: Ejecución con un receptor P3 detenido. En modo secuencial.

Como se observa en la captura, el coordinador detecta que el P3 está inalcanzable y capturo un error de comunicación. Pero también se evidencia que el tiempo de espera no fue ni siquiera cercano a los dos segundos. Esto se debe a que el coordinador al lanzar el mensaje al P3, el sistema operativo le respondio inmediatamente que el puerto estaba cerrado, teniendo un tiempo de espera similar a la prueba secuencial con tres procesos activos.

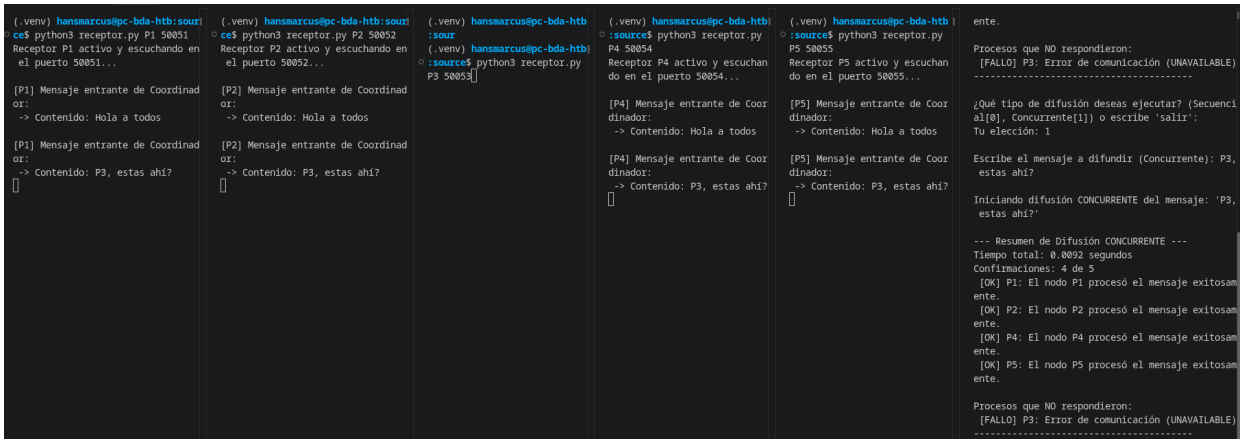


Figure 5: Ejecución con un receptor P3 detenido, el coordinador captura el tiempo que tarda en recibir la confirmación.

Aun así, se puede notar un pequeño retraso de parte del coordinador ya que al ejecutarse en modo secuencial, el hilo principal queda bloqueado hasta que se recibe la respuesta de todos los receptores, incluyendo el P3 que está inactivo y es ahí donde el tiempo de espera de 2 segundos se manifiesta.

Al ejecutar la misma prueba en modo concurrente, se observa que el tiempo de espera del coordinador disminuye significativamente. Esto se debe a que al lanzar los RPC en paralelo, el hilo principal no queda bloqueado esperando la respuesta del P3, y puede continuar con los demás receptores activos, recibiendo sus confirmaciones casi de inmediato.

3.Cambio del contenido del mensaje y verificación de que los receptores reciben el nuevo contenido.

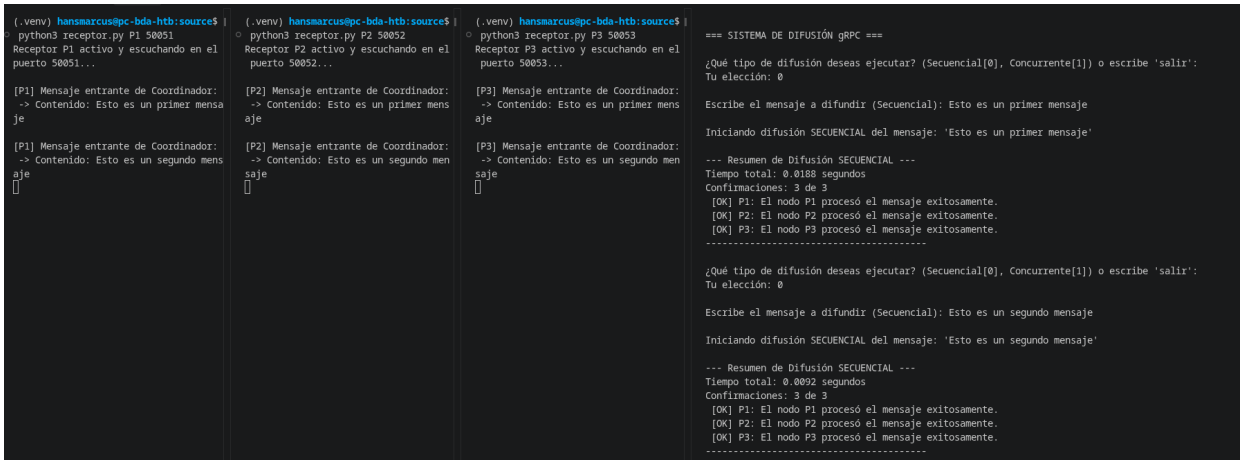


Figure 6: Envio de varios mensajes dentro de la misma sesión.

Como se observa en esta y en las pruebas anteriores, el coordinador permite enviar múltiples mensajes en una misma sesión. Los receptores muestran inmediatamente el contenido actualizado en pantalla, demostrando que la comunicación es efectiva y que los nodos están correctamente sincronizados con el coordinador.

4. Prueba con al menos cinco receptores configurados sin duplicar manualmente la lógica del servidor.



Figure 7: Ejecución con cinco receptores activos en modo secuencial.

Dado la lógica de configuración dinámica implementada en el coordinador, es posible iniciar cinco receptores sin necesidad de duplicar manualmente la lógica del servidor. Cada receptor se identifica con un ID único y escucha en un puerto distinto, permitiendo que el coordinador los contacte de manera eficiente.

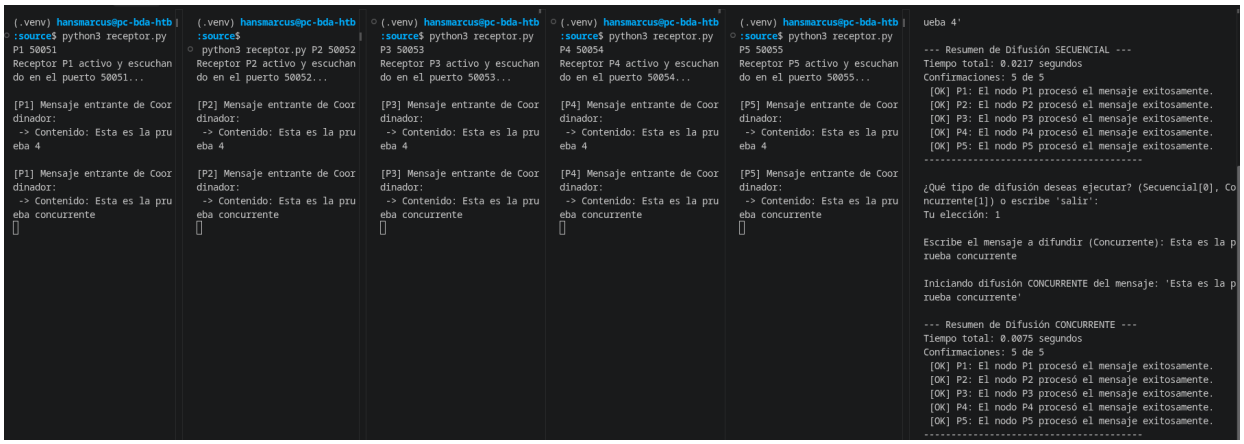


Figure 8: Ejecución con cinco receptores activos en modo concurrente.

Similar a la anterior prueba, pero ejecutada en modo concurrente. Se observa una reducción

significativa en el tiempo de latencia al enviar los RPC en paralelo, demostrando la eficiencia del enfoque concurrente incluso con un mayor número de receptores.

5. Ejecución secuencial y, como extensión obligatoria para el análisis final, ejecución concurrente de las llamadas; registrar y comparar los tiempos.

A continuación se detalla la matriz de pruebas ejecutadas, donde se contrasta el comportamiento secuencial versus el concurrente, y se explica el manejo de los fallos simulados requeridos por el proyecto.

Escenario de Prueba	Tiempo Secuencial	Tiempo Concurrente	Observaciones / Fallos Simulados
Difusión exitosa a 3 receptores activos.	≈ 0.0199 s	≈ 0.0083 s	Todos los nodos recibieron el mensaje y devolvieron la confirmación. La versión concurrente disminuyó el tiempo de latencia al lanzar los RPC en paralelo.
Receptor P3 desconectado (Fallo simulado).	≈ 0.0210 s	≈ 0.0092 s	Como se justifico, en modo secuencial el coordinador si espera los dos segundos antes de permitir al cliente mandar un nuevo mensaje. Mientras que en modo concurrente, al capturar de forma inmediata el error de p3, este permite enviar un nuevo mensaje sin esperar el timeout.
Cambio de contenido de mensaje en ejecución.	N/A	N/A	Se ingresaron mensajes distintos mediante teclado en una misma sesión. Los receptores mostraron el texto actualizado inmediatamente en pantalla.
Difusión a 5 receptores activos.	≈ 0.0217 s	≈ 0.0075 s	Todos los nodos recibieron el mensaje y devolvieron la confirmación. La versión concurrente disminuyó el tiempo de latencia al lanzar los RPC en paralelo.

Table 1: Matriz de pruebas evaluando conectividad, tolerancia a fallos y contraste de tiempos de ejecución.

Conclusión

El desarrollo y las pruebas de este proyecto demostraron que gRPC abstrae exitosamente la complejidad de la programación tradicional en red con *sockets*, permitiendo invocar procedimientos en procesos remotos con la misma naturalidad y facilidad que una función local.

El uso de Protocol Buffers garantizó un contrato estricto y tipado entre el coordinador y los receptores, asegurando la integridad de las estructuras de datos transmitidas. Además, las capacidades nativas de gRPC para la captura de excepciones (`grpc.RpcError`) y la configuración de tiempos de espera máximos (*timeouts*) resultaron fundamentales para dotar al sistema de tolerancia a fallos parciales, permitiendo que el coordinador asimilara la caída abrupta de un nodo sin interrumpir ni corromper la ejecución global del programa.

Por otro lado, el análisis y la comparativa de rendimiento comprobaron contundentemente las ventajas operativas de un enfoque concurrente frente a un modelo secuencial en sistemas distribuidos. En la ejecución secuencial, el coordinador se ve obligado a esperar pasivamente la respuesta o el fallo de un receptor antes de contactar al siguiente, lo que provoca una acumulación lineal de latencia que bloquea el hilo principal. En contraste, la implementación concurrente mediante `ThreadPoolExecutor` lanza las peticiones RPC de manera paralela hacia toda la topología.

Como se evidenció en la matriz de resultados, esta paralelización redujo drásticamente el tiempo total de difusión, pasando de ≈ 0.0217 s a ≈ 0.0075 s al contactar a 5 receptores activos. Más allá de la velocidad, el manejo concurrente demostró ser crucial para la resiliencia del sistema: al independizar las llamadas, el retraso provocado por el *timeout* de un nodo inaccesible ya no penaliza a los demás, garantizando que los receptores vivos obtengan el mensaje de forma inmediata. Esto consolida una arquitectura de red verdaderamente eficiente, robusta y lista para escalar.

Ligas al repositorio y video de demostración

- Repositorio GitHub:

1 Bibliografía

[1] Sáenz García, E. K. (2026). Notas de clase: Sistemas Distribuidos – Tema 4. Exclusión Mutua